

INTERNSHIP WEEKLY PROGRESS REPORT

Shravani More

AI Chatbot for Customer Support · AI/ML Internship Program

NextGen Forge Technologies

Candidate	Shravani More	Reference No.	NFGT/HR/INT/2026/138
College	St Francis De Sales College	Education	B.Sc
Industry Guide	Akash Jha	Reporting Manager	Sonia Sharma
Internship Duration	16 May – 17 June 2026	Mode	Remote

Week 1 Report — 18 May to 22 May 2026

Report Date	22 May 2026
Week Number	Week 1 of 4
Phase	Setup, Knowledge Base & NLP Pipeline
Hours Logged	~10 hours (2 hrs/day, Mon–Fri)
Status	Completed

Task W1.1 — Environment Setup & Knowledge Base

- Created Google Colab notebook: 01_setup_and_data.ipynb with clean Crisp DM-style markdown structure covering Business Understanding, Objective, Data Understanding sections
- Installed all required Python libraries: nltk, scikit-learn, flask, joblib
- Downloaded NLTK corpora: punkt (tokeniser), stopwords (filler word list), wordnet (lemmatisation dictionary), punkt_tab
- Created GitHub repository: ai-customer-support-chatbot with full folder structure: data/, models/, api/, interface/, notebooks/
- Github link - <https://github.com/Shravani-1325/Ai-Customer-Support-Chatbot.git>
- Created requirements.txt listing all project dependencies with version pins
- Built intents.json knowledge base with 7 intents.

Intents defined in intents.json:

Total intents: 7

- > greetings : 14 patterns, 4 responses
- > order_status : 15 patterns, 4 responses
- > return_policy : 17 patterns, 5 responses
- > billing_inquiry : 16 patterns, 4 responses
- > store_hours : 15 patterns, 4 responses
- > product_pricing : 15 patterns, 4 responses
- > escalate_to_human : 15 patterns, 4 responses

Task W1.2 — NLP Preprocessing Pipeline

- Built NLPProcessor class in Python with 3 methods: `clean_text`, `preprocess_text()` , `main()`
- **`clean_text()`** – Cleans raw text by removing punctuation, digits, and extra spaces.
- **`preprocess_text()`** – Performs tokenization, stopwords removal, and lemmatization.
- **`main()`** – Acts as the complete preprocessing pipeline by combining text cleaning and preprocessing steps.
- Tested pipeline on 10 sample queries: 7 intent samples(all caps, symbols, short input)
- Saved NLPProcessor as standalone `nlp_processor.py` module for further process.
- Documented before/after results for all 10 queries with observations table in Jupyter notebook

Week 2 Report — 25 May to 29 May 2026

Report Date	22 May 2026
Week Number	Week 2 of 4
Phase	Feature Extraction & Model Training
Hours Logged	~10 hours (2 hrs/day, Mon–Fri)
Status	Completed

Task W2.1 — TF-IDF Feature Extraction & Dataset Preparation

Converted all preprocessed text patterns into numerical feature vectors using TF-IDF, prepared train/test split, and saved the fitted vectoriser.

- Reloaded NLPProcessor class in new notebook 02_model_training.ipynb — main() method updated to return joined string instead of list (required by TfidfVectorizer)
- Loaded intents.json and extracted 107 patterns with 107 matching labels using nested loop (one label per pattern — correct supervised learning structure)
- Verified class distribution: all 7 intents present with 15–17 patterns each
- **Total patterns extracted: 107**
- **Total Labels extracted: 107**
- **Unique intent classes: 7**
- **Class Distribution:**
- **greetings : 14 patterns**
- **order_status : 15 patterns**
- **return_policy : 17 patterns**
- **billing_inquiry : 16 patterns**
- **store_hours : 15 patterns**
- **product_pricing : 15 patterns**
- **escalate_to_human : 15 patterns**
-
- Applied TfidfVectorizer(max_features=500, ngram_range=(1,2)) — ngram_range=(1,2) captures single words AND two-word phrases for richer features
- Feature matrix shape: 107 rows × 500 columns (107 patterns, each as 500-dimensional vector)
- Converted feature matrix to DataFrame for visual inspection of TF-IDF scores
- Train/test split: 80% train (85 samples) / 20% test (22 samples) with stratify=labels for proportional class distribution
- Saved fitted vectoriser as tfidf_vectorizer.pkl using joblib — verified reload (vocabulary size: 500)

Task W2.2 — Model Training & Evaluation

Trained two classifiers using a model dictionary approach, evaluated both, compared results, and saved the best model

- Created a models dictionary with Naive Bayes (MultinomialNB, alpha=0.1) and Logistic Regression (max_iter=1000, C=1.0) — cleaner than training each separately
- Built reusable model_eval() function: fits model, predicts on test set, returns accuracy, classification_report, weighted F1 score, and confusion matrix
- Iterated over models dictionary — trained and evaluated both models in a single loop
- Generated per-intent classification report for each model showing precision, recall, F1 per class

- Built results comparison DataFrame showing accuracy side-by-side for both models
- Fixed train_test_split variable order bug: correct order is x_train, x_test, y_train, y_test (not x_train, y_train, x_test, y_test)
- Used len() for y_train/y_test (Python lists) and .shape for x_train/x_test (sparse matrices)
- Saved best performing model as intent_classifier.pkl using joblib
- Ran sanity check: fed one raw query through full pipeline (preprocess → vectorise → predict) and verified correct intent output

	Model	Accuracy	F1 Score
0	Naive Bayes	0.818182	0.805519
1	Logistic Regression	0.772727	0.771429

Week 3 Report — 1 June to 5 June 2026

Report Date	22 May 2026
Week Number	Week 3 of 4
Phase	Flask API & Chat Interface
Hours Logged	~10 hours (2 hrs/day, Mon–Fri)
Status	Completed

Task W3.1 — Flask Chatbot API Development

Built the complete Flask REST API that serves ML model predictions through a clean, validated POST /chat endpoint.

- Created api/app.py with CORS enabled for cross-origin browser requests (flask-cors)
- Loaded intent_classifier.pkl, tfidf_vectorizer.pkl, intents.json at server startup — not per-request, to avoid 0.5s loading overhead on every message
- **POST /chat**: receives {message} JSON → NLP preprocess → TF-IDF vectorise → predict intent → random response from intents.json → return {response, intent, confidence}
- **GET /health**: returns server status — used for deployment health checks on Render/Railway
- **Input validation**: returns HTTP 400 if message field is missing or empty
- **Confidence threshold**: queries below 0.15 return 'I did not understand' fallback instead of a wrong intent response
- Absolute file paths via os.path.abspath(__file__) — API works from any directory
- Used absolute file paths via os.path.abspath(__file__) so API works regardless of which directory it is run from
- NLPProcessor class pasted directly into app.py for reliability — avoids import path issues across environments
- Tested all endpoints in Thunder Client (VS Code extension): 10+ test cases covering all 9 intents, empty message, missing field, health check
- Tested in Thunder Client (VS Code): 10+ test cases covering all 9 intents, empty message, missing field, health check
- Total patterns extracted: 140
- Total Labels extracted: 140
- Unique intent classes: 9
-
- Class Distribution:
- greetings : 14 patterns
- order_status : 15 patterns
- return_policy : 17 patterns
- billing_inquiry : 16 patterns
- store_hours : 15 patterns
- product_pricing : 15 patterns
- escalate_to_human : 15 patterns
- chitchat : 16 patterns
- shipping_delay : 17 patterns

API endpoint summary:

Method	Endpoint	Input	Output
POST	/chat	{"message": "..."}	{"response":"...", "intent":"...", "confidence":0.94}
GET	/health	none	{"status":"ok", "message":"Chatbot API is running"}

Task W3.2 — Chat Interface Development

Built a production-quality single-page glassmorphism chat interface connecting to Flask API via JavaScript `fetch()`.

- `interface/chat.html` — single file, no external frameworks, plain HTML/CSS/JavaScript
- Design: glassmorphism card over Unsplash background with purple-blue gradient overlays and backdrop-filter blur
- Hero header 'NovaMind Support' with candidate name, college and reference number byline
- Distinct message bubbles: user (purple tint) vs bot (glass), with avatars and timestamps on every message
- Intent tag shown under each bot response showing live ML classification (e.g. BILLING INQUIRY, ORDER STATUS)
- Typing indicator: 3 animated bouncing dots while waiting for API response
- Suggestion chips: 4 quick-start buttons that disappear after first use
- Error toast notification if Flask server is unreachable
- REST API integration: single `fetch()` POST call — sends `{message}`, reads `data.response` and `data.intent` from JSON reply

Deliverable	Details	Status
<code>api/app.py</code>	Flask API, 2 endpoints, validation, confidence threshold	✓ Done
<code>interface/chat.html</code>	Glassmorphism chat UI, full API integration	✓ Done
Thunder Client tests	10+ test cases, all intents + error scenarios	✓ Done
GitHub push	<code>api/</code> and <code>interface/</code> folders committed	✓ Done

Model Retrained Results

	Model	Accuracy	F1 Score
0	Logistic Regression	0.750000	0.704337
1	Naïve Byes	0.678571	0.613265

Challenges — Week 3

- 404 on `http://127.0.0.1:5000/` — Flask only responds to defined routes, not root URL. Fixed by visiting `/health` directly
- `InconsistentVersionWarning` on `pkl` load (sklearn 1.6.1 Colab vs 1.5.2 local) — fixed by upgrading local sklearn
- CORS error blocking browser `fetch()` calls — fixed with `flask-cors` and `CORS(app)`

- 500 error after retraining — old.pkl files loaded (231 features vs 303 expected). Fix: always restart Flask after saving new.pkl files
- Pattern overlap between intents is the most common cause of misclassification in small NLP datasets. When two intents share vocabulary (store_hours + escalate both using "available"), the model can't distinguish them
- Further New Intents Added - **chitchat : 16 patterns , shipping_delay : 17 patterns**
- **Data Insufficiency & Its Impact:** The training dataset for this project consisted of 140 patterns across 9 intent classes, averaging approximately 15 patterns per intent — which is considered minimal for a production-grade NLP classifier. When two new intents (chitchat and shipping_delay) were introduced mid-project, the already limited data became further distributed, causing class imbalance and vocabulary overlap between similar intents. This directly resulted in misclassifications, for example, store_hours patterns being predicted as escalate_to_human due to shared words like "available" and "reach." The model required three full retraining iterations with pattern rewrites to reach a stable 75% accuracy. A dataset of 500+ patterns per intent would significantly reduce such overlap issues, improve model generalisation to unseen user inputs, and produce a more confident and reliable classifier.

Week 4 Report — 8 June to 12 June 2026

Report Date	22 May 2026
Week Number	Week 4 of 4
Phase	Deployment Testing
Hours Logged	~10 hours (2 hrs/day, Mon–Fri)
Status	Completed

Task W4.1 — Cloud Deployment & End-to-End Testing

Deployed the Flask chatbot API to Render free tier with live public URL. Resolved multiple deployment-specific issues through independent research and problem solving.

Deployment Preparation

- Created Procfile in project root: web: gunicorn api.app:app — tells Render how to start the application
- Updated app.py final line: reads PORT from environment variable (os.environ.get('PORT', 5000)), set host='0.0.0.0', debug=False for production
- Updated requirements.txt: removed strict version pins, added gunicorn and setuptools for production compatibility

Render Deployment Configuration

- Created Render Web Service connected to GitHub repo: Shravani-1325/Ai-Customer-Support-Chatbot
- Settings: Region Singapore, Branch main, Build Command pip install -r requirements.txt, Start Command gunicorn api.app:app
- Health Check Path set to /health — Render pings this endpoint to monitor server uptime
- Auto-Deploy set to On Commit — every GitHub push triggers automatic redeployment

Issues Encountered & Resolved

ISSUE 1

Build failed: Cannot import setuptools.build_meta. Root cause: Render defaulted to Python 3.14 which had no pre-built wheels for pinned numpy/scikit-learn versions. Fix: Created runtime.txt forcing Python 3.11.9, removed version pins from requirements.txt.

ISSUE 2

405 Method Not Allowed when visiting /chat URL in browser. Root cause: Browser makes GET requests, /chat only accepts POST. Initial fix was to use Thunder Client for testing. Final fix: served chat.html directly from Flask so the live URL renders the full UI.

RESOLVED

Researched and implemented proper Flask template serving: configured template_folder to point to interface/ directory, added GET / route using render_template('chat.html'), updated API_URL in chat.html to relative path '/chat' instead of hardcoded localhost URL. Result: visiting https://novamind-chatbot.onrender.com/ now renders the full glassmorphism chat UI directly.

Live Deployment Verification

Test	URL	Result
Frontend UI loads	https://novamind-chatbot.onrender.com/	✔200 OK
Health check	https://novamind-chatbot.onrender.com/health	✔200 OK
Chat endpoint	POST /chat {message: 'where is my order'}	✔200 OK
Intent classification	9 intents tested via live URL	✔Correct
Error handling	Empty message field	✔400 Error

Task W4.2 — Documentation & Project Submission

README.md

- Project overview and problem statement
- Complete intent list with example patterns for all 9 intents
- Local setup instructions: clone, virtualenv, pip install, python api/app.py
- API reference: POST /chat and GET /health with sample request/response JSON
- Live deployment URL and note about free tier spin-down behaviour
- Model training summary: dataset size, algorithm used, accuracy achieved
- Future improvements section: more training data, additional intents, transformer models

Future Improvements (Documented in README)

- Expand to 500+ patterns per intent using data augmentation and synonym replacement for significantly better accuracy
- Add intents: shipping_inquiry, account_issue, password_reset, promotional_offers, product_complaint
- Replace TF-IDF + Logistic Regression with a transformer-based model (BERT, DistilBERT) for context-aware classification
- Add conversation memory so the bot can reference previous messages in the same session
- Upgrade to paid hosting to eliminate the 50-second cold start on Render free tier

Week 4 Deliverables

Deliverable	Details	Status
Procfile	gunicorn api.app:app	Done
runtime.txt	Python 3.11.9 specified	Done
Live Render URL	https://novamind-chatbot.onrender.com	Live
Frontend on live URL	chat.html served via Flask render_template()	Done
README.md	Full documentation with setup, API ref, URL	Done
Weekly Progress Report	All 4 weeks documented	Done
Final submission email	GitHub link + live URL sent to HR	Done

Challenges — Week 4

- PowerShell not recognising git commands — resolved by switching VS Code terminal to Git Bash

- Render defaulting to Python 3.14 causing build failure — resolved with runtime.txt pinning Python 3.11.9
- Package version conflicts on Render — resolved by removing strict version pins from requirements.txt
- 405 Method Not Allowed on /chat in browser — resolved by serving chat.html from Flask and using relative API paths
- Free tier spin-down (50 second cold start) — documented in README as known limitation, noted as future improvement